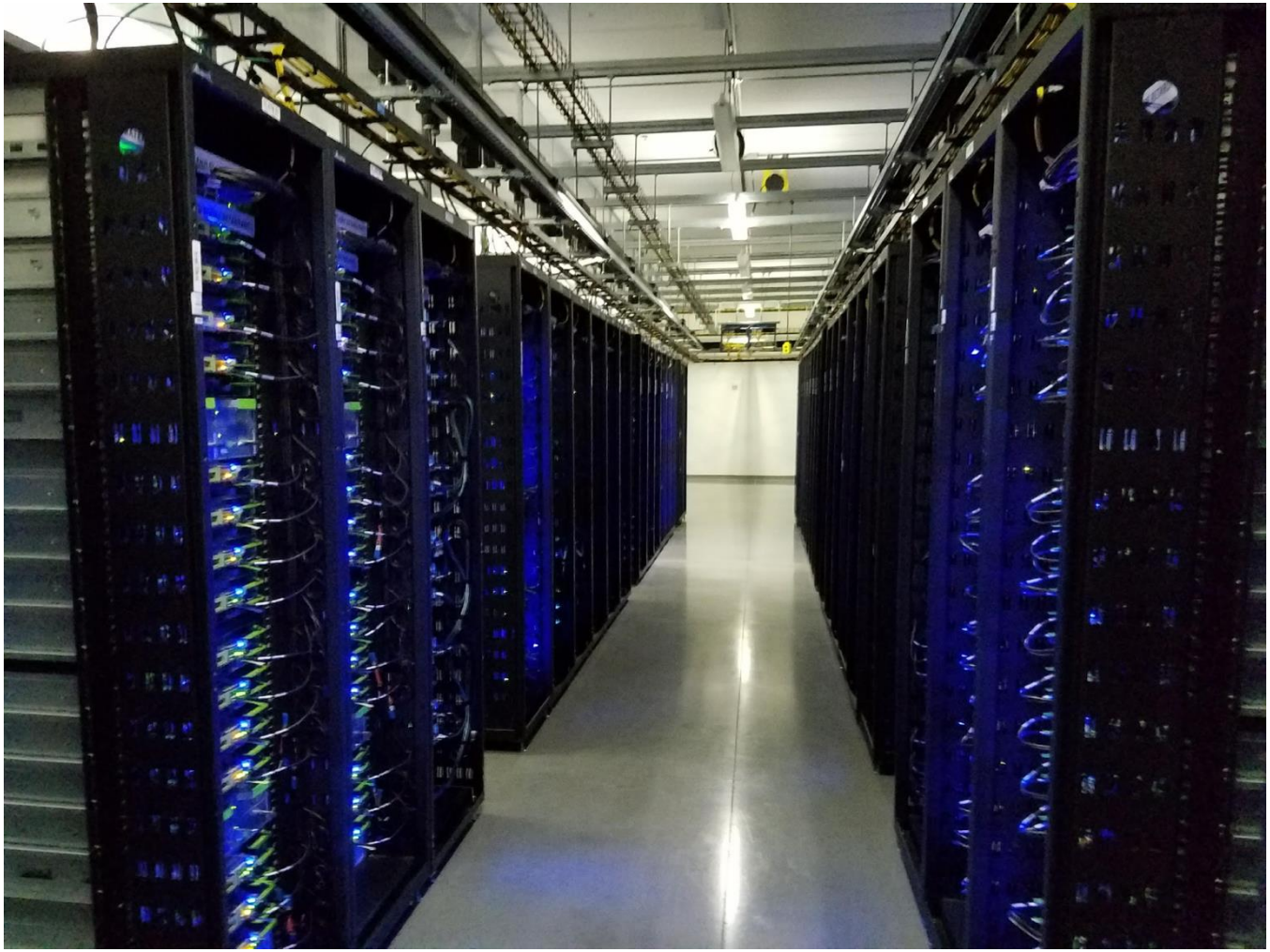
MapReduce



Some slides by Andrew DeOrio

Agenda

- Distributed systems introduction
  - MapReduce motivation
- MapReduce programming model
- MapReduce execution model
- Example
- Fault tolerance

Distributed systems

- Distributed system: Multiple computers cooperating on a task
- MapReduce: Distributed system for compute
  - Run a program that would be too slow on one computer
  - Today
- Google File System: Distributed system for storage
  - Store more data than fits on one computer
  - Next time

Big program examples

- Counting the number of words in 1 TB of text files
- Count number of requests to each web page, given a log
- Building a search index over large amount of files

Not a distributed system

- Write a program to count the frequency of words using Python

```
$ cat input.txt
Hello World
Bye World
Hello Hadoop
Goodbye Hadoop

$ cat input.txt | python3 wc.py
Hello 2
World 2
Bye 1
Hadoop 2
Goodbye 1
```

Not a distributed system

```
import sys
import collections

word_count = collections.defaultdict(int)

for line in sys.stdin:
    words = line.split()
    for word in words:
        word_count[word] += 1

for word, count in word_count.items():
    print(word + "\t" + count)
```

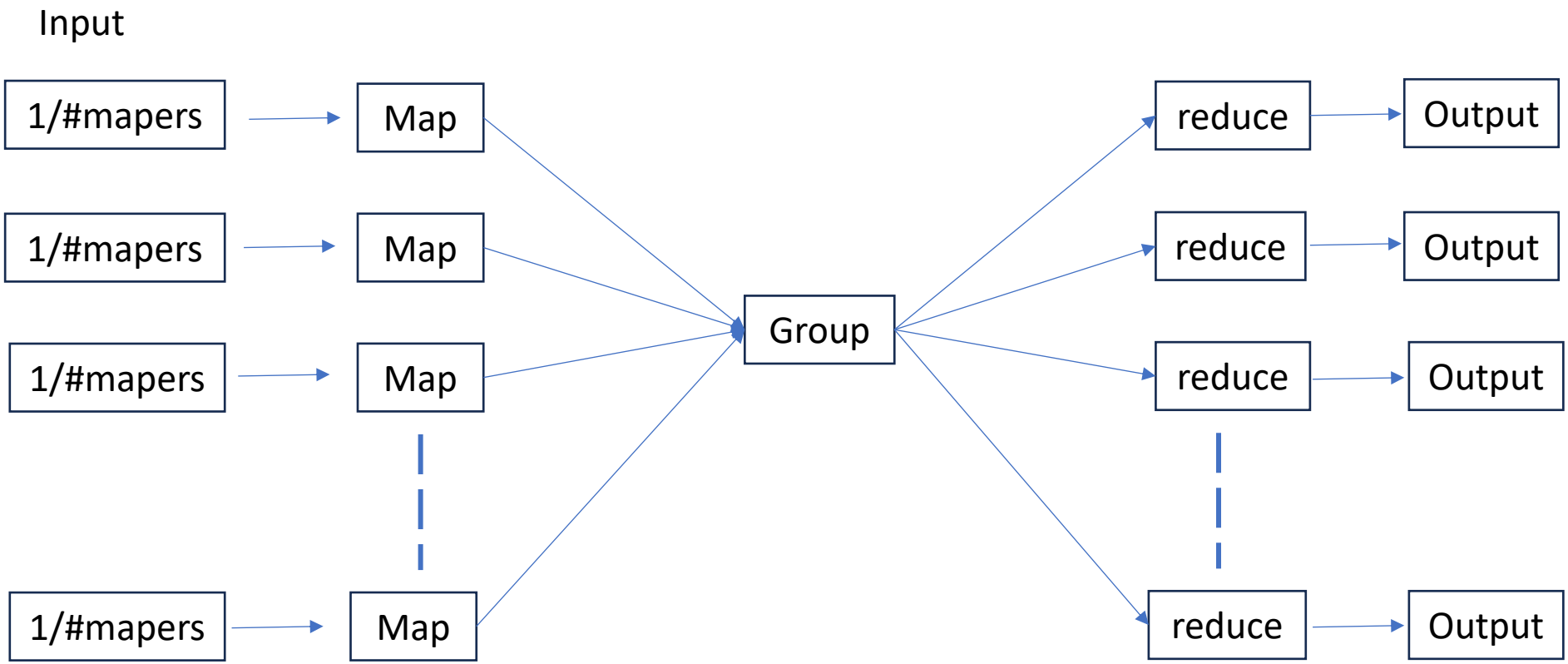
`defaultdict` automatically initializes values to zero

MapReduce model

- Three phases
- **Map**: turn input into (key, value) pairs
- **Group**: put entries with the same key together
- **Reduce**: combine adjacent (key, value) pairs into a final answer

MapReduce picture

MapReduce picture



MapReduce: Map

```
import sys
import collections

word_count = collections.defaultdict(int)

for line in sys.stdin:
    words = line.split()
    for word in words:
        word_count[word] += 1

for word, count in word_count.items():
    print(word + "\t" + count)
```

`defaultdict` automatically initializes values to zero

MapReduce: Map

- Split input into two files
- Run same program on two computers

```
word_count = collections.defaultdict(int)
for line in sys.stdin:
    words = line.split()
    for word in wors:
        word_count[word] += 1
```

input01.txt

```
Hello World Bye
World
```

```
word_count = collections.defaultdict(int)
for line in sys.stdin:
    words = line.split()
    for word in wors:
        word_count[word] += 1
```

input02.txt

```
Hello Hadoop
Goodbye Hadoop
```

- Problem: Two computers can't share a data structure

MapReduce: Map

- Two computers, two data structures, two outputs
- Problem: need to "put together" the output

```
word_count = collections.defaultdict(int)
for line in sys.stdin:
    words = line.split()
    for word in wors:
        word_count[word] += 1
```

input01.txt
Hello World Bye
World

map

output

Hello	**1**
World	2
Bye	1

```
word_count = collections.defaultdict(int)
for line in sys.stdin:
    words = line.split()
    for word in wors:
        word_count[word] += 1
```

input02.txt
Hello Hadoop
Goodbye Hadoop

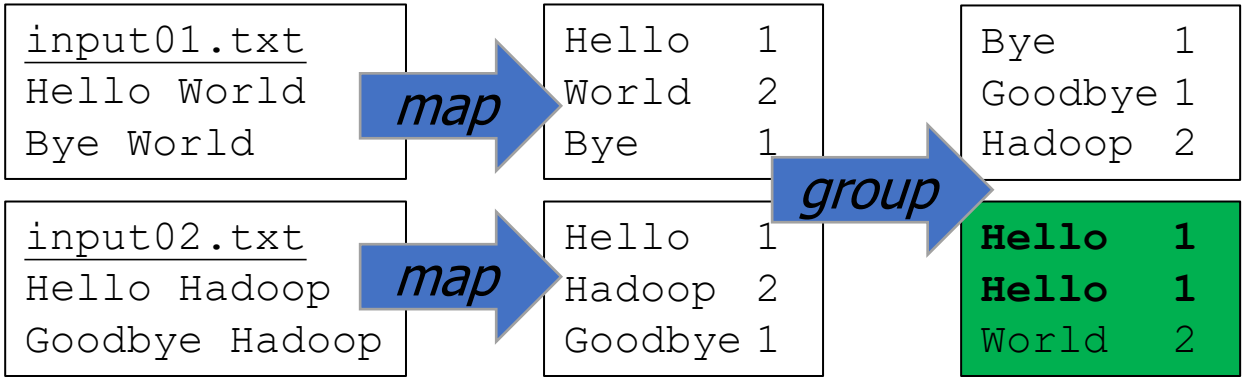
map

output

Hello	**1**
Hadoop	2
Goodbye	1

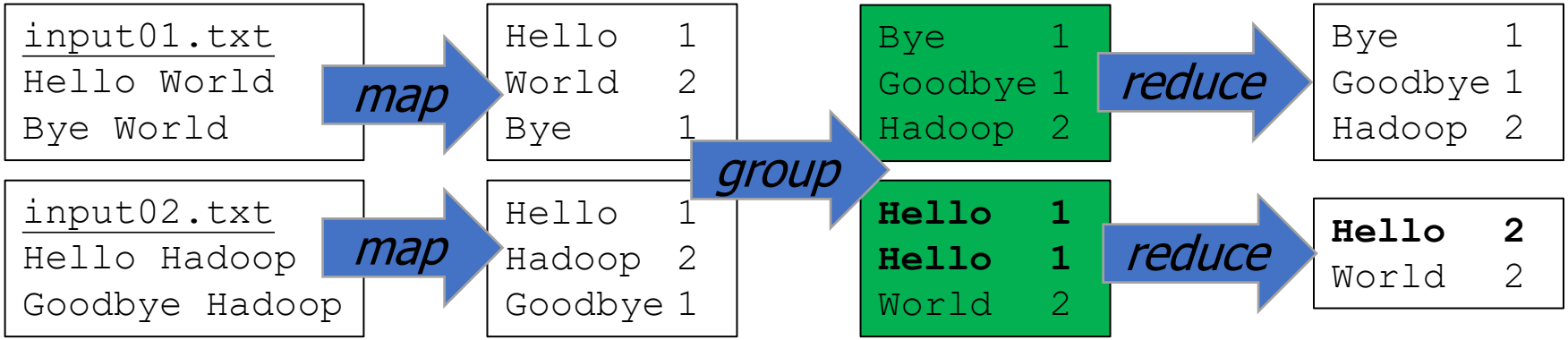
MapReduce: Group

- Problem: need to "put together" the output
- Insight: output of first program(s) is in the form *<key> <value>*
- Solution: every line with the **same key** is in the **same group**
- This is called *grouping*



MapReduce: Reduce

- After we have groups, we can "put together the output"
- This is called *reducing*
  - Runs in parallel, just like map
- Write this program



MapReduce: Reduce

- In this example, the second program is very similar to the first
- No shared data structures

```
Bye     1
Goodbye 1
Hadoop  2
```

```
word_count = collections.defaultdict(int)
for line in sys.stdin:
    word, count = line.split()
    word_count[word] += count
for word, count in word_count.items():
    print(word + "\t" + count)
```

```
Bye     1
Goodbye 1
Hadoop  2
```

```
Hello   1
Hello   1
World   2
```

```
word_count = collections.defaultdict(int)
for line in sys.stdin:
    word, count = line.split()
    word_count[word] += count
for word, count in word_count.items():
    print(word + "\t" + count)
```

```
Hello   2
World   2
```

New application: Web access log stats

- Count the number of visits to each page in TBs of server logs
- Solution is a lot like the previous word count example

```
server1-nginx.log
2021-05-10 09:51 141.213.74.63 index.html 200
2021-05-10 09:52 35.7.44.69 index.html 403
```

```
server2-nginx.log
2021-05-10 10:05 141.213.74.63 syllabus.html 200
2021-05-10 11:19 35.7.44.69 p1-insta485-static.html 403
```

New application and old application

- Old application: word count
- New application: web page access count

- Some code will be different
  - Read input format
- Some code will be identical
  - The communication and coordination between machines

- We'd have to solve all the same parallelization problems again!
  - Enter MapReduce

MapReduce

- MapReduce Framework handles
  - Parallelization over many machines
  - Segment and distribute input
  - Fault tolerance
- MapReduce provides a clean abstraction for programmers
- Programmer writes two programs
  - Map
  - Reduce

Agenda

- Distributed systems introduction
  - MapReduce motivation
- **MapReduce programming model**
- MapReduce execution model
- Example
- Fault tolerance

MapReduce programming model

- Our examples will use the **Hadoop** MapReduce streaming interface

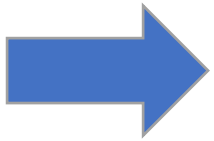
- Map is a program
  - Each line of input is from an input file
  - Each line of output is `<key>\t<value>`
- Group is provided by MapReduce framework
- Reduce is a program
  - Each line of input is `<key>\t<value>`
  - Each line of output is up to the programmer

A second look at word count

- Count the number of occurrences of each word in a collection of documents
- We'll solve the same problem in a slightly different way

```
input01.txt
Hello World
Bye World
```

```
input02.txt
Hello Hadoop
Goodbye Hadoop
```



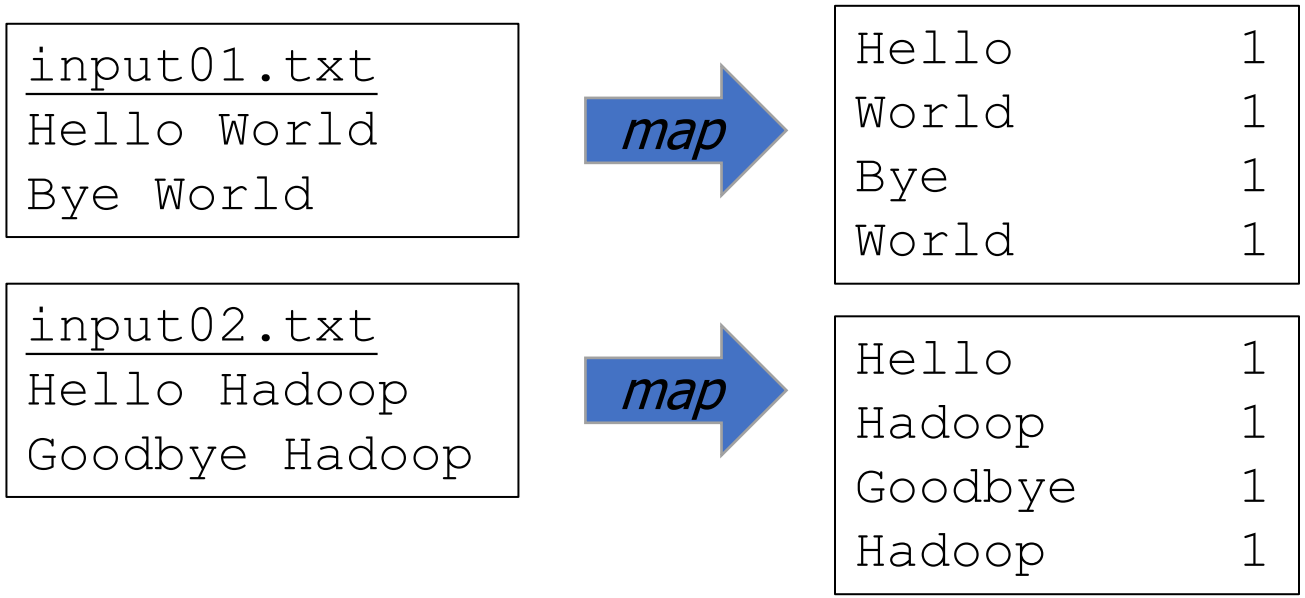
output

```
Bye          1
Goodbye      1
Hadoop       2
Hello        2
World        2
```

Word count map

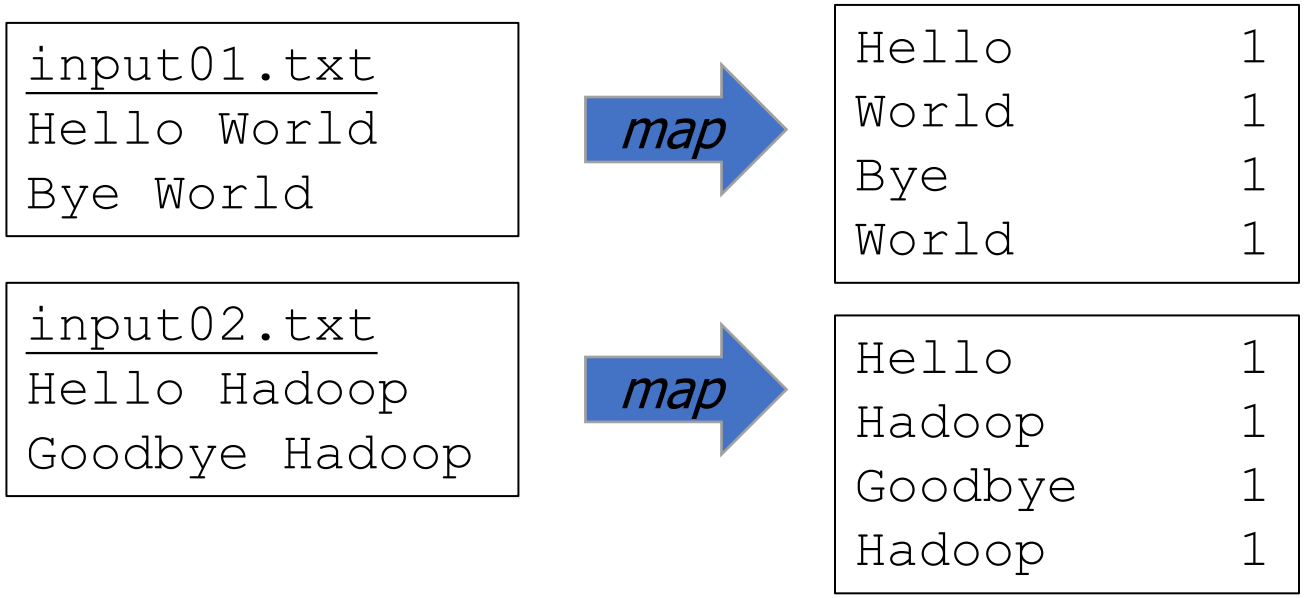
```
import sys
for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print(word + "\t" + "1")
```

- A mapper doesn't count words, it just splits them up



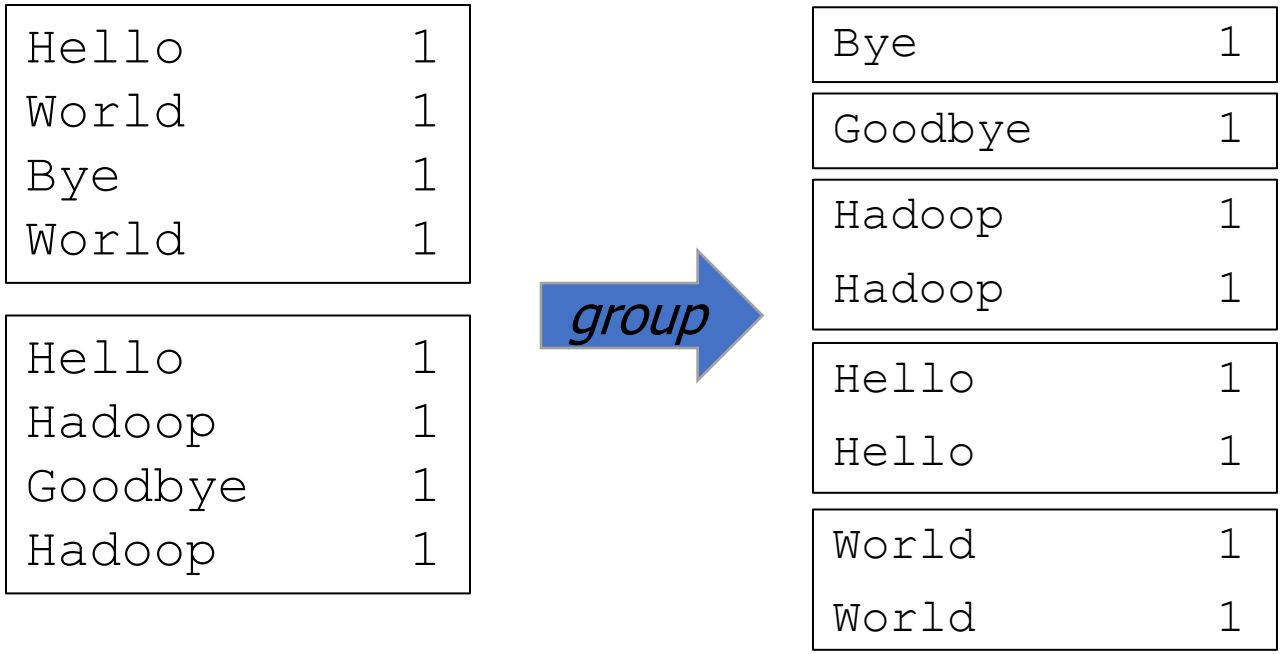
Intermediate key value pairs

- Map output is in the form of *key value pairs*
- Separated by a TAB in the Hadoop streaming interface
- What to put here is up to the programmer



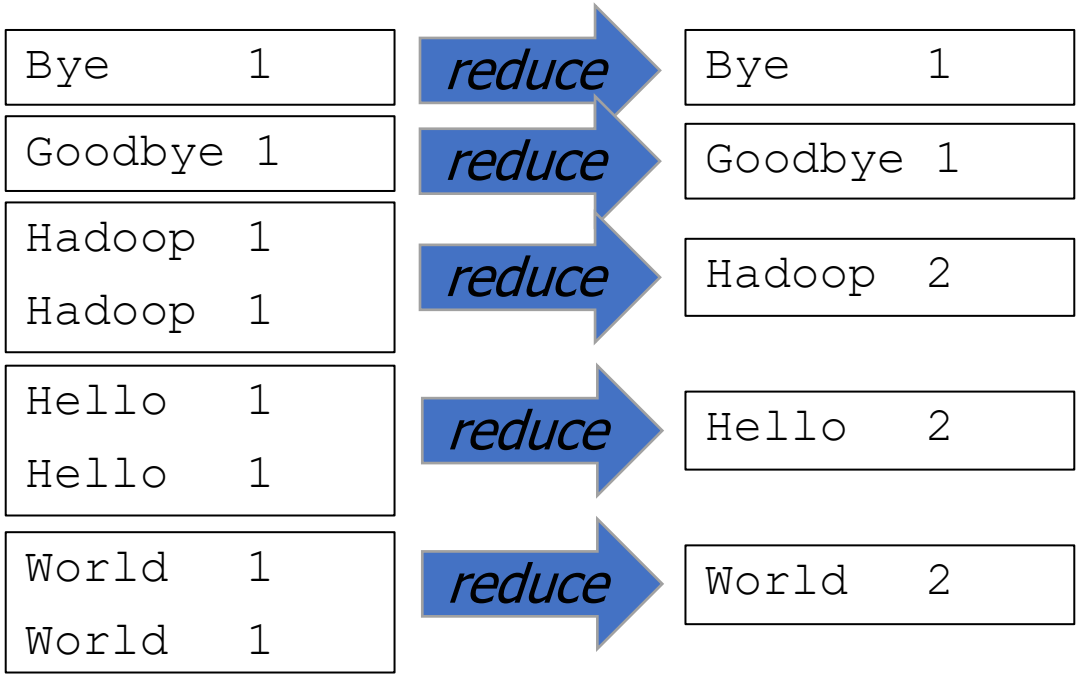
Word count group

- Group is provided by the MapReduce framework
- Group by key



Word count reduce

```
counts = collections.defaultdict(int)

for line in sys.stdin:
    line = line.strip()
    word, count = line.split("\t")
    counts[word] += int(count)

for word, count in counts.items():
    print(word + "\t" + str(count))
```



Intermediate data size

- Isn't the intermediate data size large?
- Option 1: a "local reducer" called a Combiner at each map
  - Compresses data between map and reduce
- Option 2: a different map implementation

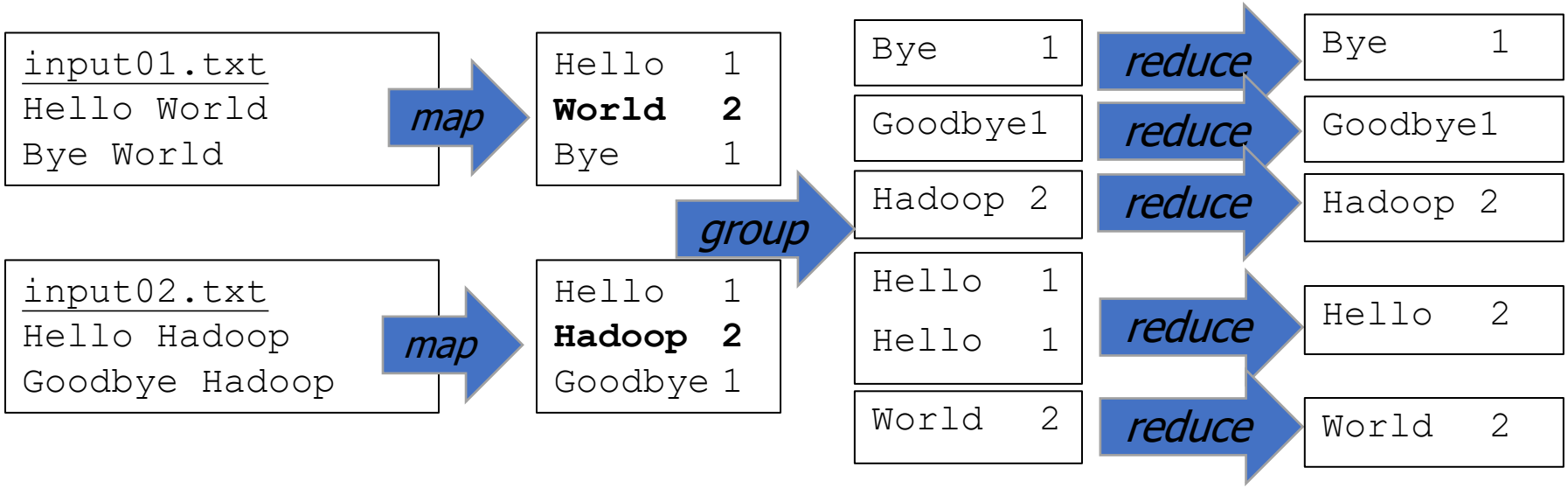
Word count

- Option 2: map sums the words in each doc
  - This is our previous implementation

```
import collections
import sys

word_count = collections.defaultdict(int)

for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
    word_count[word] += 1

for word, count in word_count.items():
    print(word + "\t" + str(count))
```

Word count

```
word_count = collections.defaultdict(int)

for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
    word_count[word] += 1

for word, count in word_count.items():
    print(word + "\t" + str(count))
```



same answer

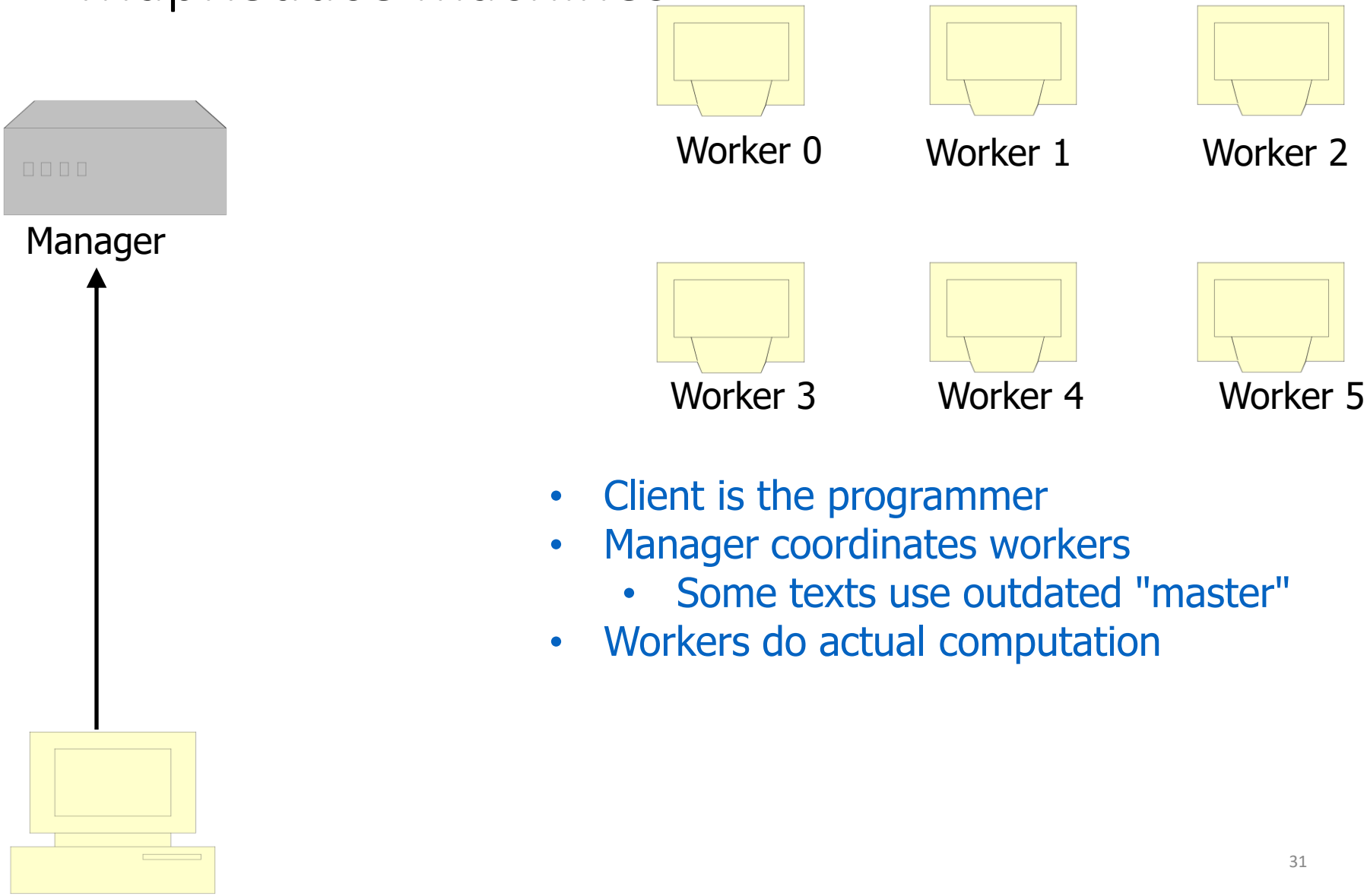
Agenda

- Distributed systems introduction
  - MapReduce motivation
- MapReduce programming model
- **MapReduce execution model**
- Example
- Fault tolerance

MapReduce execution model

- Segment input
- Map stage
- Group stage
- Reduce stage

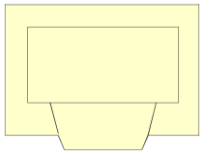
MapReduce machines



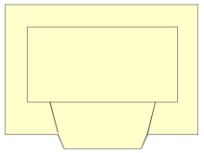
- Client is the programmer
- Manager coordinates workers
  - Some texts use outdated "master"
- Workers do actual computation

Job processing

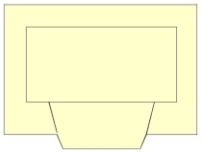
Manager



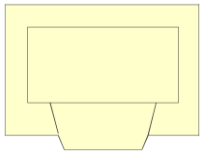
Worker 0



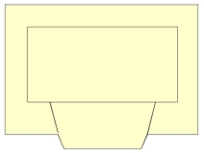
Worker 1



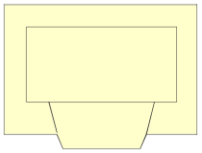
Worker 2



Worker 3

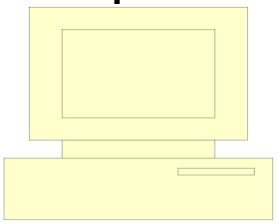


Worker 4



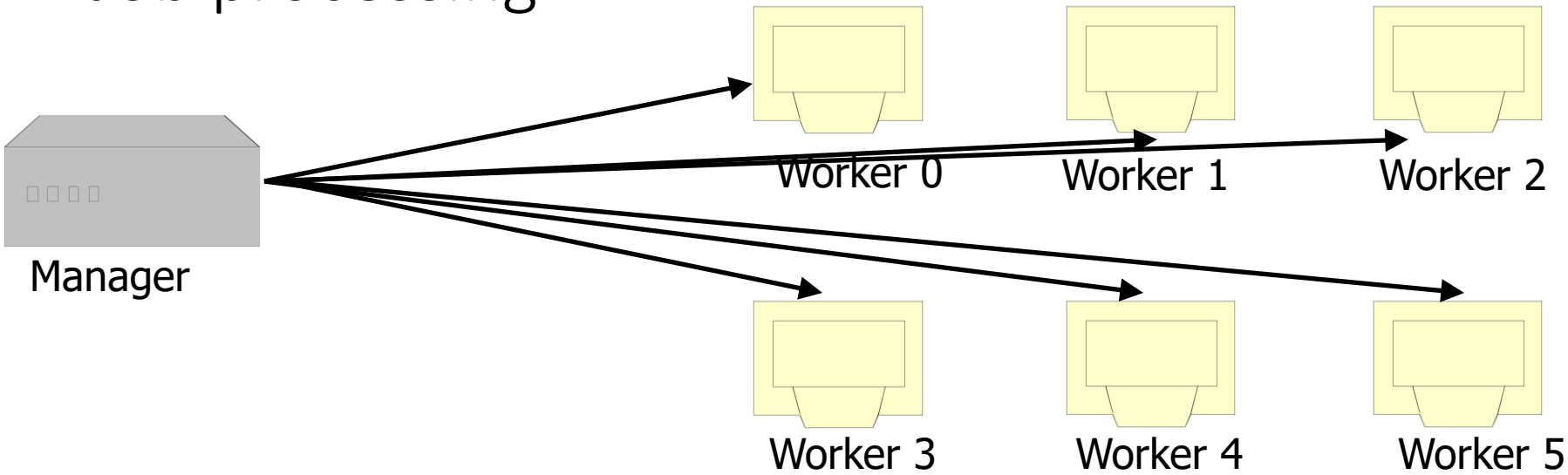
Worker 5

```
{
  "mapper": "wc_map.sh",
  "reducer": "wc_reduce.sh"
}
```

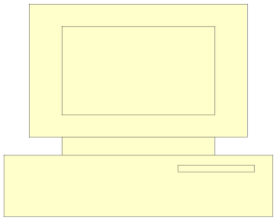


1. Client submits word count job, indicating map code, reduce code and input files

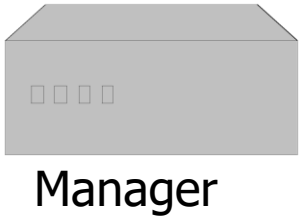
Job processing

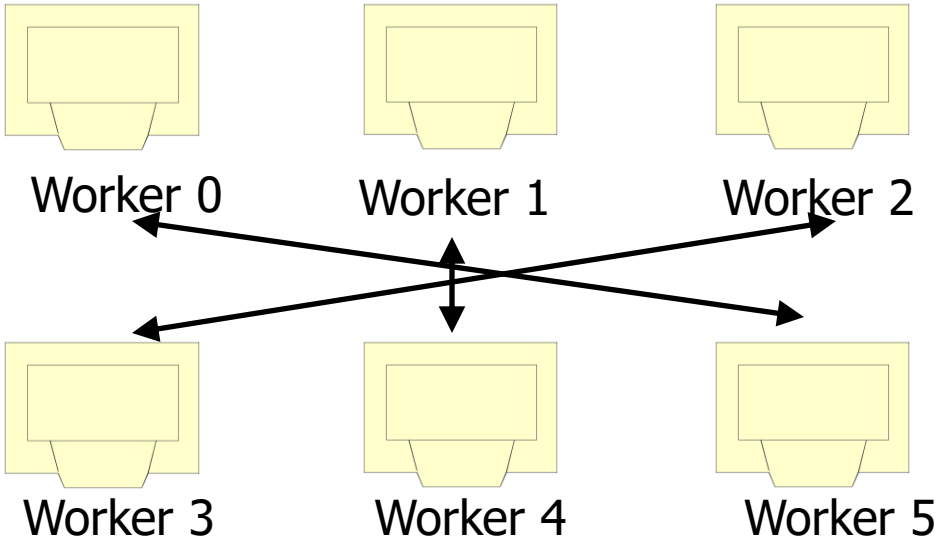


1. Client submits word count job, indicating map code, reduce code and input files
2. Manager breaks input file into *k* chunks, (in this case 6). Assigns work to workers.

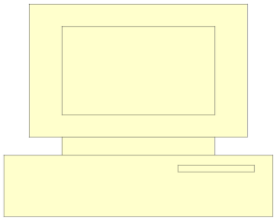


Job processing

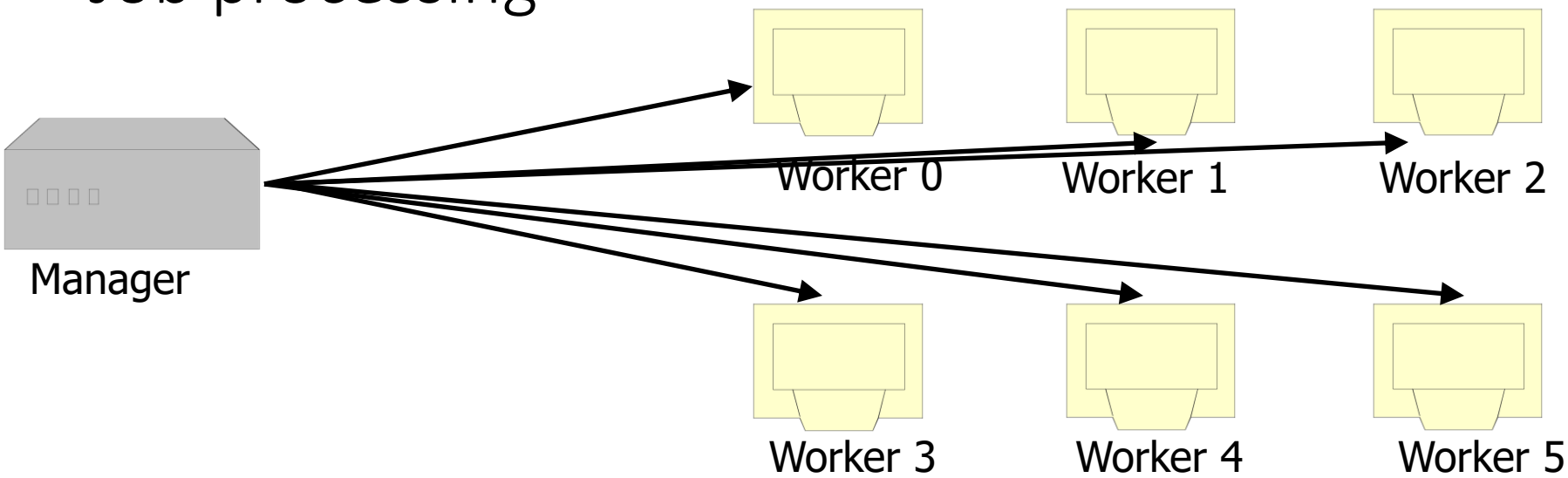




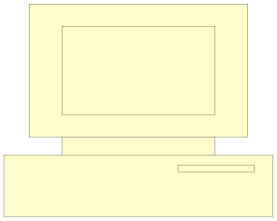
1. Client submits word count job, indicating map code, reduce code and input files
2. Manager breaks input file into *k* chunks, (in this case 6). Assigns work to workers.
3. After map(), workers exchange map-output to produce groups



Job processing

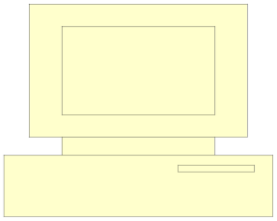


1. Client submits word count job, indicating map code, reduce code and input files
2. Manager breaks input file into *k* chunks, (in this case 6). Assigns work to workers.
3. After map(), workers exchange map-output to produce groups
4. Manager reduce() keyspace into *m* chunks (in this case 6). Assigns work.



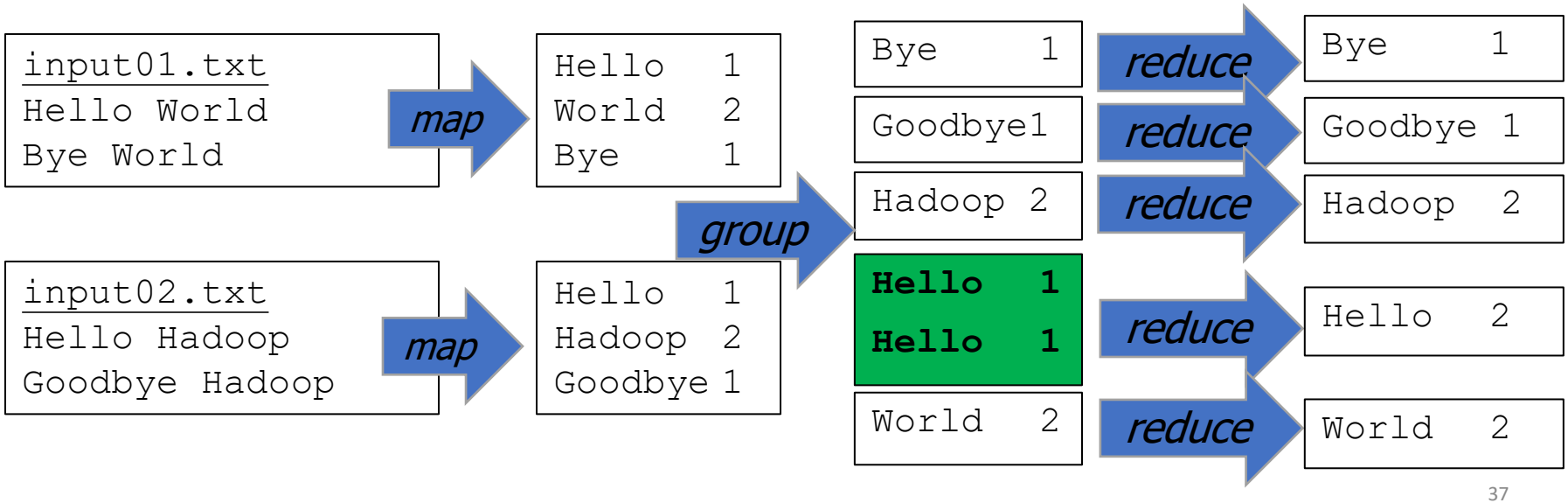
Job processing

1. Client submits word count job, indicating map code, reduce code and input files
2. Manager breaks input file into *k* chunks, (in this case 6). Assigns work to workers.
3. After map(), workers exchange map-output to produce groups
4. Manager breaks reduce() keyspace into *m* chunks (in this case 6). Assigns work.
5. reduce() output may go to shared drive



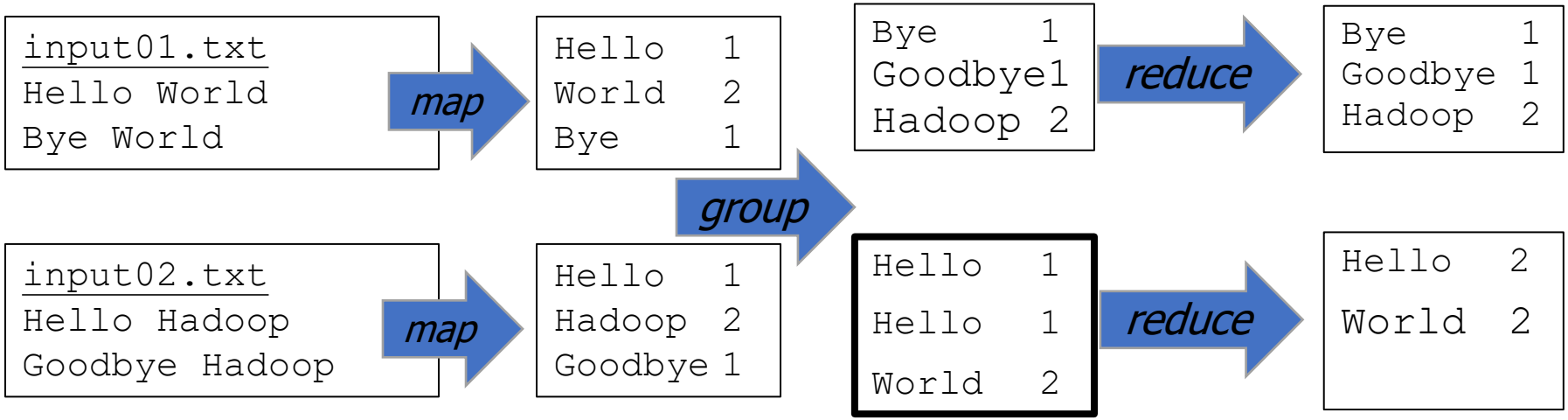
Same key, same group

- MapReduce framework provides grouping functionality
- All values for the **same key** *must* be in the **same group**



Same key, same group

- Could a group have more than 1 key? Yes.
  - In the Hadoop streaming interface



Project 4

- You'll write a MapReduce framework in EECS 485 Project 4
- We'll provide the MapReduce programs

Agenda

- Distributed systems introduction
  - MapReduce motivation
- MapReduce programming model
- MapReduce execution model
- **Example**
- Fault tolerance

Web access logs

- Input: many TBs of server logs
- Output: Which IPs got 403 responses? How many?

- Input

```
2021-05-10 09:51 141.213.74.63 index.html 200
2021-05-10 09:52 35.7.44.69 index.html 403
```

```
2021-05-10 10:05 141.213.74.63 syllabus.html 200
2021-05-10 11:19 35.7.44.69 p1-insta485-static.html 403
2021-10-05 09:33 35.7.44.21 p3-clientside.html 403
```

- Output

```
35.7.44.21 1
```

```
35.7.44.69 2
```

Web access log code

- Map

```
for line in sys.stdin:
    date, time, ip, path, code = line.strip().split()
    if code == "403":
        print(f"{ip}\t1")
```

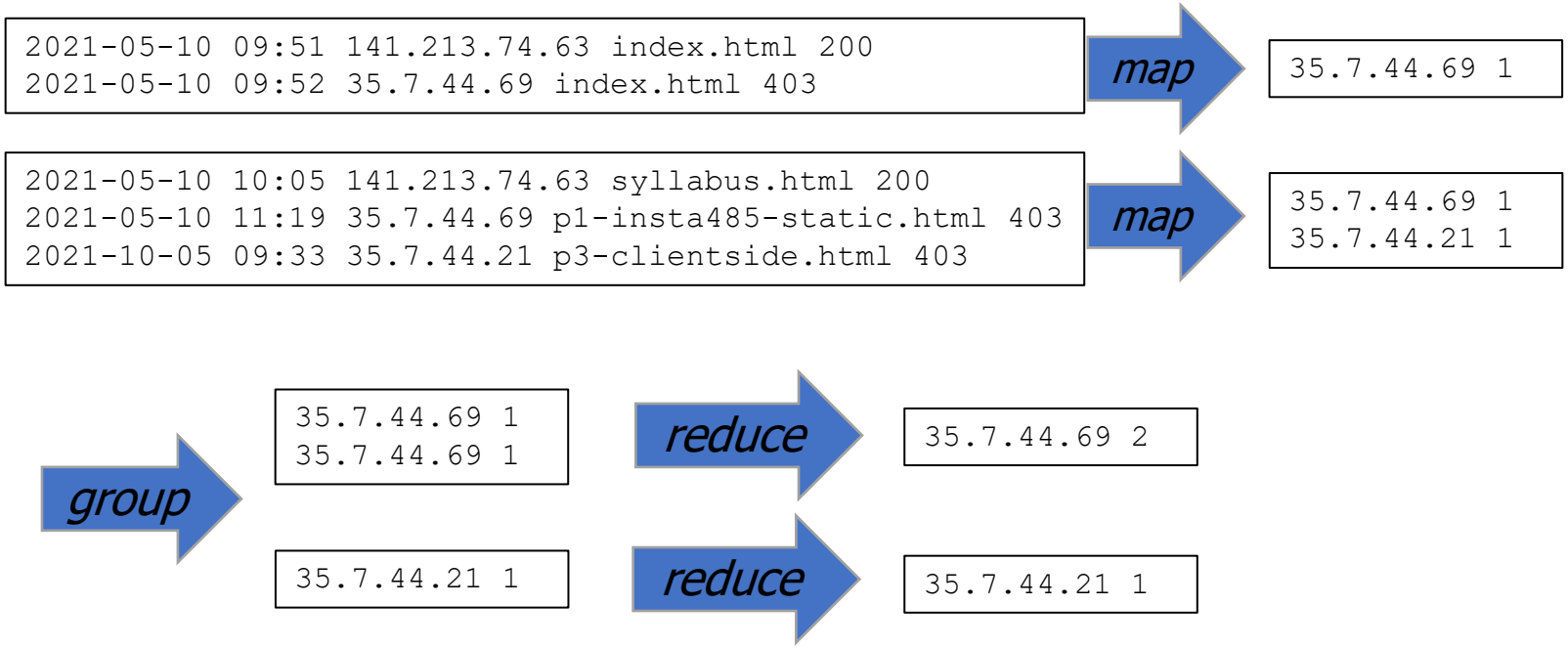
- Reduce

```
ip_count = collections.defaultdict(int)
for line in sys.stdin:
    ip, count = line.strip().split("\t")
    ip_count[ip] += int(count)

for ip, count in ip_count.items():
    print(f"{ip}\t{count}")
```

Web access log diagram

```
map.py
for line in sys.stdin:
    date, time, ip, path, code = line.strip().split()
    if code == "403":
        print(f"{ip}\t1")
```

```
reduce.py
ip_count = collections.defaultdict(int)
for line in sys.stdin:
    ip, count = line.strip().split("\t")
    ip_count[ip] += int(count)

for ip, count in ip_count.items():
    print(f"{ip}\t{count}")
```



Agenda

- Distributed systems introduction
  - MapReduce motivation
- MapReduce programming model
- MapReduce execution model
- Example
- **Fault tolerance**

Fault tolerance

- How do we know if a machine goes down?

- Workers send periodic heartbeat messages to Manager
- Manager keeps track of which workers are up

Fault tolerance

- What happens when a machine dies?
- Without MapReduce
  - Program (or query) is restarted
  - Not so hot if your job is in hour 23
- With MapReduce
  - If map worker dies
    - Just restart that task on a different box
    - You lose the map work, but no big deal
  - If reduce worker dies
    - Restart the reducer, using output from source mappers

Fault tolerance nice side effects

- What about slow, not dead, machines?
- Speculative execution for stragglers
- Kill the 2nd-place finisher

Further reading

- Nice explanation from UC Berkeley
  - http://inst.eecs.berkeley.edu/~cs61a/book/chapters/streams.html#distributed-data-processing
- Some researchers disagree with MapReduce's popularity: “MapReduce: A Major Step Backwards”
  - https://homes.cs.washington.edu/~billhowe/mapreduce_a_major_step_backwards.html
- Paper on Google's MapReduce framework "MapReduce: Simplified Data Processing on Large Clusters" by Jeffrey Dean and Sanjay Ghemawat
  - https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf